



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE3101 Bases de Datos

Documentación Técnica

Proyecto NutriTEC

Integrantes:

Luis Eladio Castro — 2013083843

Gabriel Soto López — 2024178797

Ariel Saborio Alvarez — 2024101248

Sofia Xie Xie — 2024100513

Profesor: Marco Rivera Meneses

Primer Semestre 2026

23 de junio, 2026

Tabla de contenidos

1. Introducción	3
2. Modelo conceptual	4
3. Modelo relacional.....	4
3.1 Mapeo de tipo de entidades fuertes	4
3.2 Mapeo de tipo de entidades débiles.....	5
3.3 Mapeo de tipo de asociación 1:1	5
3.4 Mapeo de tipo de asociación 1:N	5
3.5 Mapeo de tipo de asociación N:M	6
3.6 Mapeo de atributos multivaluados	6
3.7 Resultado del Modelo Relacional	7
4. Estructura de datos	7
4.1 Modelos.....	7
4.2 Persistencia con SQL Server.....	8
4.3 Persistencia con MongoDB Atlas.....	9
4.4 Capa de Repositorios	10
4.5 Capa de Servicios.....	10
4.6 DTOs (Data Transfer Objects).....	10
5. Store procedures, triggers y vistas	12
5.1 Store Procedures.....	12
5.2 Triggers	14
6.3 Vistas (Views)	15
6. Arquitectura	16
6.1 Capas del sistema	16
6.2 Flujo de comunicación.....	17
6.3 Patrones de diseño	18
6.4 Diagrama de clases	18
7. Problemas conocidos.....	19
8. Problemas encontrados	20
9. Conclusiones	21
10. Recomendaciones	22
11. Bibliografía	23

1. Introducción

NutriTEC es una plataforma integral orientada a la gestión y seguimiento nutricional de usuarios, la cual contiene funciones como control y monitoreo de la información nutricional de los alimentos que se consume, visualizar el progreso físico de los pacientes, asimismo, los nutricionistas registrados pueden diseñar planes alimenticios para cada paciente. La solución permite la interacción entre clientes, nutricionistas y administradores a través de diferentes módulos especializados, facilitando la gestión centralizada de la información y el seguimiento personalizado de cada usuario.

La arquitectura de la solución está basada en una aplicación web y una aplicación móvil, respaldadas por una infraestructura de bases de datos híbrida. Para el almacenamiento de información estructurada se utiliza Microsoft SQL Server, mientras que MongoDB Atlas se emplea para la gestión de información no estructurada relacionada con la retroalimentación y comunicación entre nutricionistas y pacientes. Asimismo, la comunicación entre los diferentes componentes del sistema se realiza mediante una API desarrollada en C#, siguiendo una arquitectura orientada a servicios que favorece la escalabilidad, mantenibilidad y reutilización de los componentes.

Este documento presenta la documentación técnica del proyecto, incluyendo el análisis y diseño de la solución, el modelo conceptual y relacional de la base de datos, la arquitectura implementada, la descripción de los componentes desarrollados, los procedimientos almacenados, vistas y disparadores utilizados, así como los principales desafíos encontrados durante el desarrollo y las soluciones adoptadas. El objetivo es proporcionar una visión completa de la implementación realizada y servir como referencia para futuras tareas de mantenimiento, evolución y despliegue del sistema.

2. Modelo conceptual

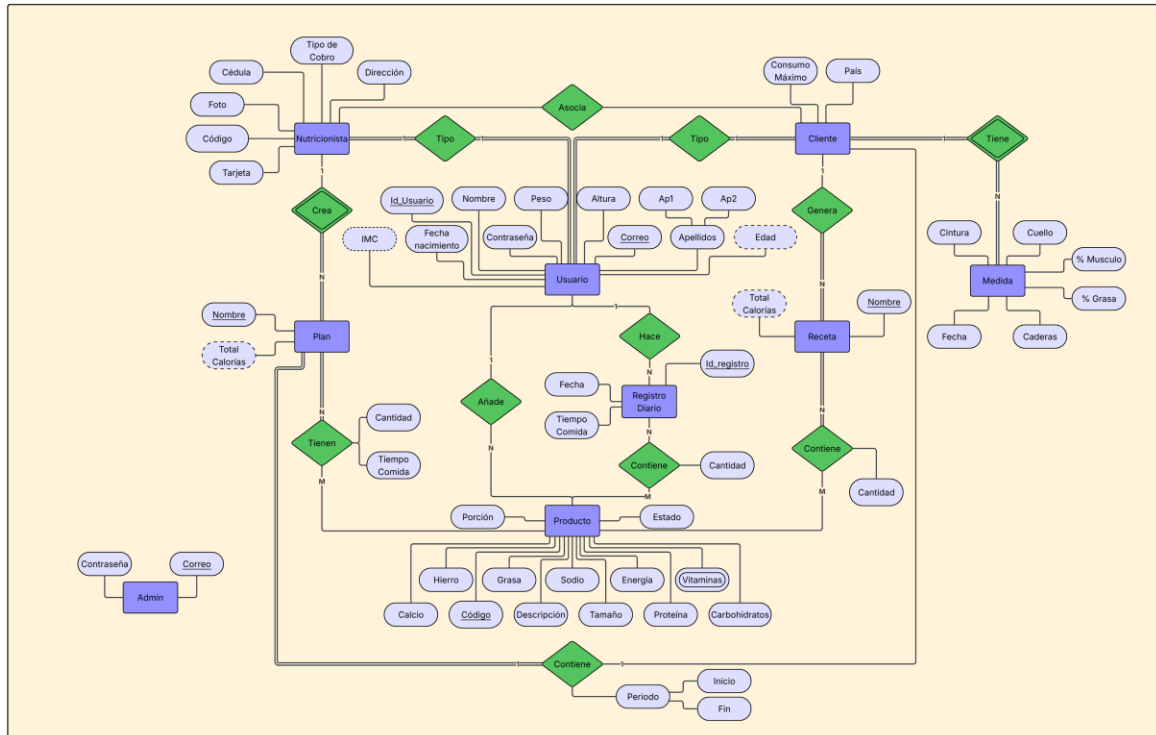


Figura 1. Modelo Conceptual

3. Modelo relacional

3.1 Mapeo de tipo de entidades fuertes

Se agregaron una relación para cada entidad fuerte del modelo conceptual, con sus respectivos atributos. Para los atributos compuestos como apellidos en la entidad usuario se insertó atributos adicionales para almacenar cada valor individualmente. Finalmente se puede observar las entidades fuertes con su llave primaria en la siguiente tabla:

Tabla 1. Entidades Fuertes

Entidad	Llave primaria
Usuario	id_usuario
Cliente	id_usuario (hereda de Usuario)
Nutricionista	id_usuario (hereda de Usuario)
Admin	Correo
Plan	id_plan
Receta	Id_receta
Registro Diario	Id_registro
Producto	Id_producto

3.2 Mapeo de tipo de entidades débiles

En este modelo no existen entidades débiles, ya que todas las entidades del diagrama conceptual poseen un atributo identificador propio (subrayado), lo cual les permite distinguirse de forma independiente. Por lo tanto, este paso no aplica y se procede directamente con las relaciones 1:1, 1:N y N:M.

3.3 Mapeo de tipo de asociación 1:1

La llave primaria de una de las dos entidades se incluye como llave foránea en la otra (normalmente en el lado que participa obligatoriamente, o se fusionan en una sola tabla). Como se puede ver en la siguiente tabla:

Tabla 2. Relaciones de 1:1

Relación	Entidad con participación	Llave foránea
Usuario → Nutricionista	Nutricionista	Id_usuario
Usuario → Cliente	Cliente	Id_usuario

3.4 Mapeo de tipo de asociación 1:N

Se identifican las relaciones que son del tipo 1 a N. Se toma la llave primaria de la entidad padre (1) y se agrega como llave foránea a la entidad hijo (N). Note que en el modelo Chen hay una relación con atributo, para eso se crea una nueva relación de asociación.

Tabla 3. Relaciones de 1:N

Relación	Entidad con participación	Llave foránea
Usuario → Producto	Producto	id_usuario
Usuario → Registro Diario	Registro Diario	id_cliente
Cliente → Receta	Receta	id_cliente
Cliente → Medida	Medida	id_usuario (Cliente)
Nutricionista → Plan	Plan	Id_nutricionista
Nutricionista → Cliente (Asocia)*	Cliente	

*Nota: la relación "Asocia" es 1:N con participación opcional del lado Cliente (un cliente puede no tener nutricionista asignado, y como máximo tiene uno). Por decisión de diseño, en lugar de usar FK con NULL en Cliente, se optó por implementarla mediante una tabla puente (ClientexNutricionista).

3.5 Mapeo de tipo de asociación N:M

Se crea una tabla nueva (tabla intermedia o "puente"), cuya llave primaria es la combinación de las llaves primarias de ambas entidades participantes. Si la relación tiene atributos propios (como Cantidad, Tiempo, etc.), estos se agregan como columnas adicionales en esa tabla intermedia.

Tabla 4. Relaciones de N:M

Relación	Tabla intermedia	Llaves foráneas	Atributos propios
Nutricionista ↔ Cliente (Asocia)	ClientexNutricionista	Cliente, Nutricionista	
Plan ↔ Producto (Tienen)	ProductoxPlan	id_plan, id_producto	Cantidad, Tiempo_comida
Producto ↔ Registro Diario (Contiene)	RegistroxProducto	id_registro, id_producto	Cantidad
Receta ↔ Producto (Contiene)	ProductoxReceta	id_receta, id_producto	Cantidad
Cliente ↔ Plan (Contiene/Periodo)	PlanxCliente	id_cliente, id_plan	Inicio, Fin

3.6 Mapeo de atributos multivaluados

Cuando un atributo es multivaluado (puede tener varios valores para una misma entidad, como las Vitaminas de un Producto), no se puede representar como una sola columna. Se crea una tabla nueva donde cada fila representa un valor del atributo, y se incluye como llave foránea la PK de la entidad a la que pertenece (en este caso, id_producto). La llave primaria de esta nueva tabla suele ser la combinación de ambos campos (Vitaminas + id_producto), ya que un producto puede tener varias vitaminas y una misma vitamina puede repetirse en distintos productos.

3.7 Resultado del Modelo Relacional

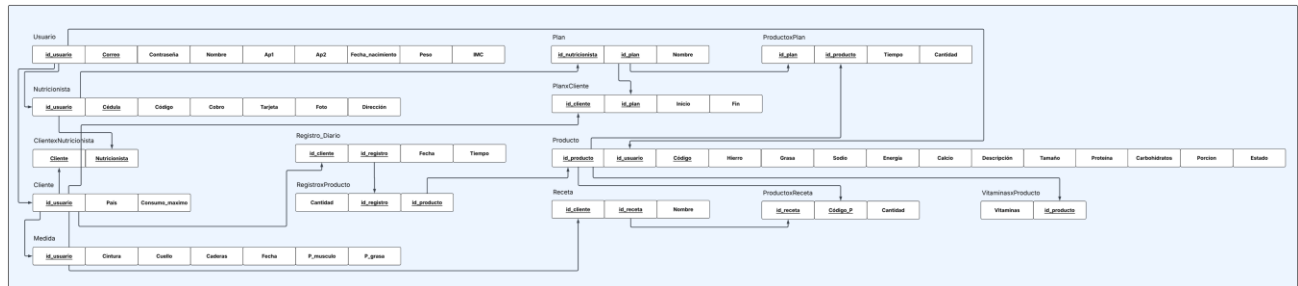


Figura 2. Modelo Relacional

4. Estructura de datos

En este apartado se describen las estructuras de datos implementadas en el sistema para el almacenamiento, administración y manipulación de la información. El sistema utiliza SQL Server como base de datos relacional principal y MongoDB Atlas para el almacenamiento del foro de retroalimentación.

4.1 Modelos

Los modelos del proyecto, ubicados en la carpeta NutriTEC.API/Models, son clases de C# que representan tanto entidades 1 a 1 del modelo relacional como objetos de negocio compuestos por múltiples tablas. Las clases implementadas son: Usuario, Admin, Nutricionista, Cliente, ClientexNutricionista, Medida, Producto, VitaminaxProducto, Receta, ProductoxReceta, PlanAlimentacion, ProductoxPlan, PlanxCliente, RegistroDiario, RegistroxProducto, y la clase Retroalimentacion (con su clase anidada RespuestaForo) para el documento de MongoDB Atlas.

A diferencia de un mapeo estrictamente 1 a 1, algunos modelos del proyecto representan objetos de negocio compuestos a partir de varias tablas relacionadas entre sí, mientras que las consultas a la base de datos se ejecutan mediante SQL explícito (FromSqlRaw / ExecuteSqlRawAsync de Entity Framework), para tener control directo sobre las consultas y poder invocar los Stored Procedures implementados.

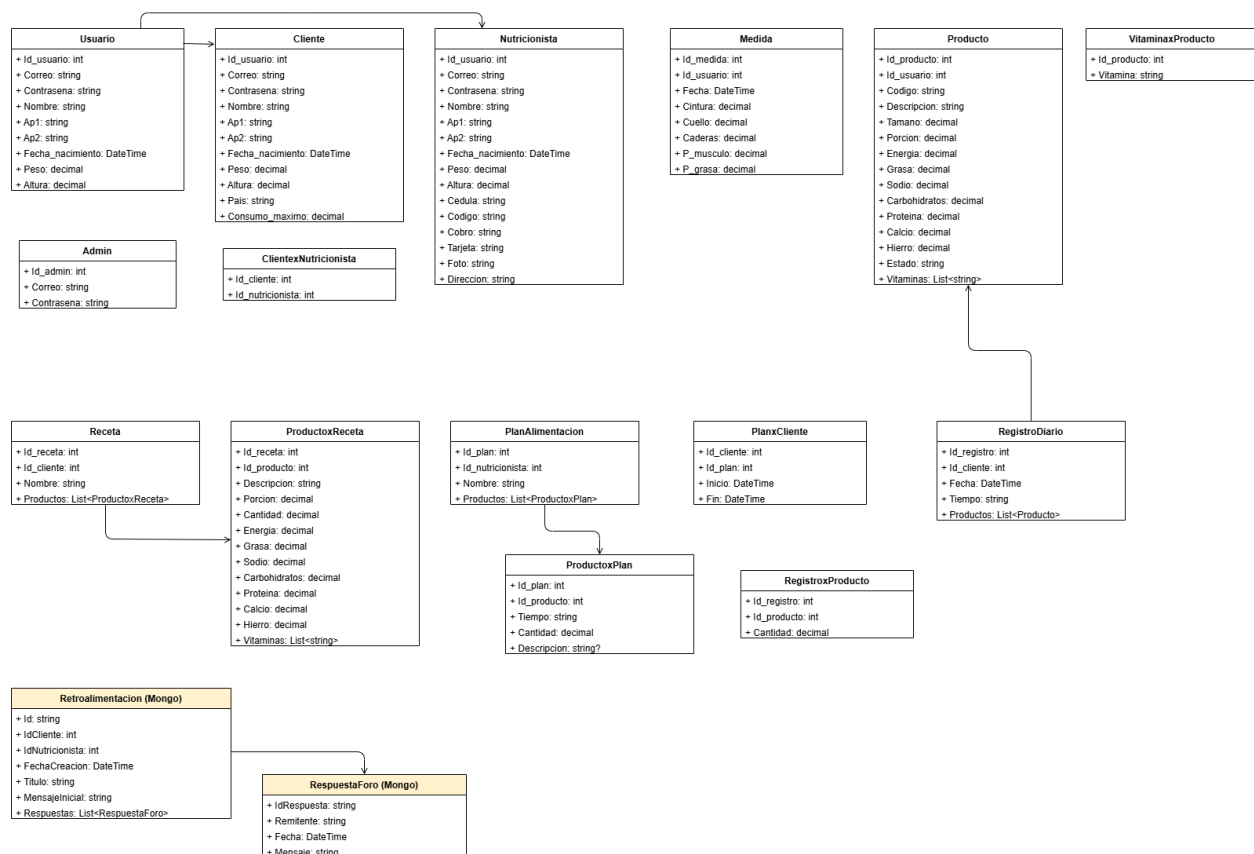


Figura 3. Modelos de las entidades.

4.2 Persistencia con SQL Server

La persistencia principal del sistema se realiza mediante SQL Server, donde cada entidad del modelo relacional corresponde a una tabla. Las operaciones de inserción, actualización, eliminación y consulta se implementan a través de Stored Procedures, garantizando que toda la lógica de acceso a datos resida en la base de datos y no en el código de aplicación. Las relaciones entre entidades se implementan con llaves primarias y foráneas para garantizar integridad referencial.

La persistencia principal del sistema se realiza mediante SQL Server (base de datos `nutritec_db`), donde cada tabla corresponde a una entidad o asociación del modelo relacional. La conexión a la base de datos se gestiona mediante Entity Framework Core, configurado en la clase `NutriTECContext` y registrado en el contenedor de inyección de dependencias de ASP.NET Core (`Program.cs`).

Para la conexión a nivel de base de datos, se creó un login y usuario específico para la aplicación (`nutritec_app`), con permisos de lectura y escritura (`db_datareader`,

db_datawriter) y permiso de ejecución de Stored Procedures (GRANT EXECUTE), siguiendo el principio de menor privilegio en lugar de utilizar la cuenta de administrador (sa) desde la aplicación.

4.3 Persistencia con MongoDB Atlas

El módulo de retroalimentación del nutricionista al paciente utiliza MongoDB Atlas, dado que la naturaleza de foro (sin límite de extensión ni de respuestas) no se adapta bien a un esquema relacional fijo. Los documentos en MongoDB Atlas tienen la siguiente estructura:

El módulo de retroalimentación entre nutricionista y paciente utiliza MongoDB Atlas, dado que la naturaleza de foro (con un número variable de respuestas y sin una estructura fija de campos) se adapta mejor a un esquema de documentos que a un esquema relacional rígido. La conexión se gestiona mediante una clase singleton (MongoDatabaseConnection), inyectada en los servicios de Cliente y Nutricionista que requieren acceder al foro.

El documento Retroalimentacion implementado tiene la siguiente estructura:

Tabla 5. Estructura del documento de Retroalimentación

Campo (BSON)	Tipo	Descripción
_id	ObjectId	Identificador único del documento, generado por MongoDB Atlas
id_cliente	int	Referencia (FK lógica) al cliente en SQL Server
id_nutricionista	int	Referencia (FK lógica) al nutricionista en SQL Server
fecha_creacion	DateTime	Fecha y hora de creación del hilo (UTC)
titulo	string	Asunto del hilo de retroalimentación
mensaje_inicial	string	Contenido del mensaje inicial
respuestas	array de RespuestaForo	Lista de respuestas al hilo, sin límite de extensión

Cada elemento del arreglo respuestas (subdocumento RespuestaForo) contiene: id_respuesta (GUID generado por la aplicación), remitente ("Cliente" o "Nutricionista"), fecha (UTC) y contenido (texto de la respuesta). Este diseño permite que un hilo de retroalimentación crezca de forma flexible sin necesidad de modificar el esquema ni de realizar JOINS adicionales, a diferencia de lo que ocurriría en un modelo relacional equivalente.

4.4 Capa de Repositorios

Para desacoplar la capa de acceso a datos de la lógica de negocio, se definió un repositorio por cada controlador funcional del sistema, ubicados en NutriTEC.API/Data/Repositories: AuthRepository, NutricionistaRepository, ClienteRepository, ProductoRepository, PlanRepository, RecetaRepository, AdminRepository y MedidaRepository. Cada repositorio encapsula las consultas SQL (incluyendo la invocación de Stored Procedures) necesarias para su entidad correspondiente, y los repositorios de Cliente y Nutricionista incorporan además el acceso a la colección de retroalimentación en MongoDB Atlas a través de MongoDBDatabaseConnection.

4.5 Capa de Servicios

Sobre la capa de repositorios se implementó una capa de servicios (NutriTEC.API/Services) que concentra la lógica de negocio: AuthService, NutricionistaService, ClienteService, ProductoService, PlanService, RecetaService, AdminService y MedidaService. Por ejemplo, AuthService es responsable de validar credenciales contra el hash almacenado con BCrypt, determinar el rol del usuario autenticado (nutricionista, cliente o administrador) y generar el token JWT correspondiente.

4.6 DTOs (Data Transfer Objects)

Los DTOs son objetos utilizados para transferir información entre las distintas capas del sistema cuando la información requerida no corresponde directamente a una única entidad del modelo. A continuación, se presentan los DTO implementados:

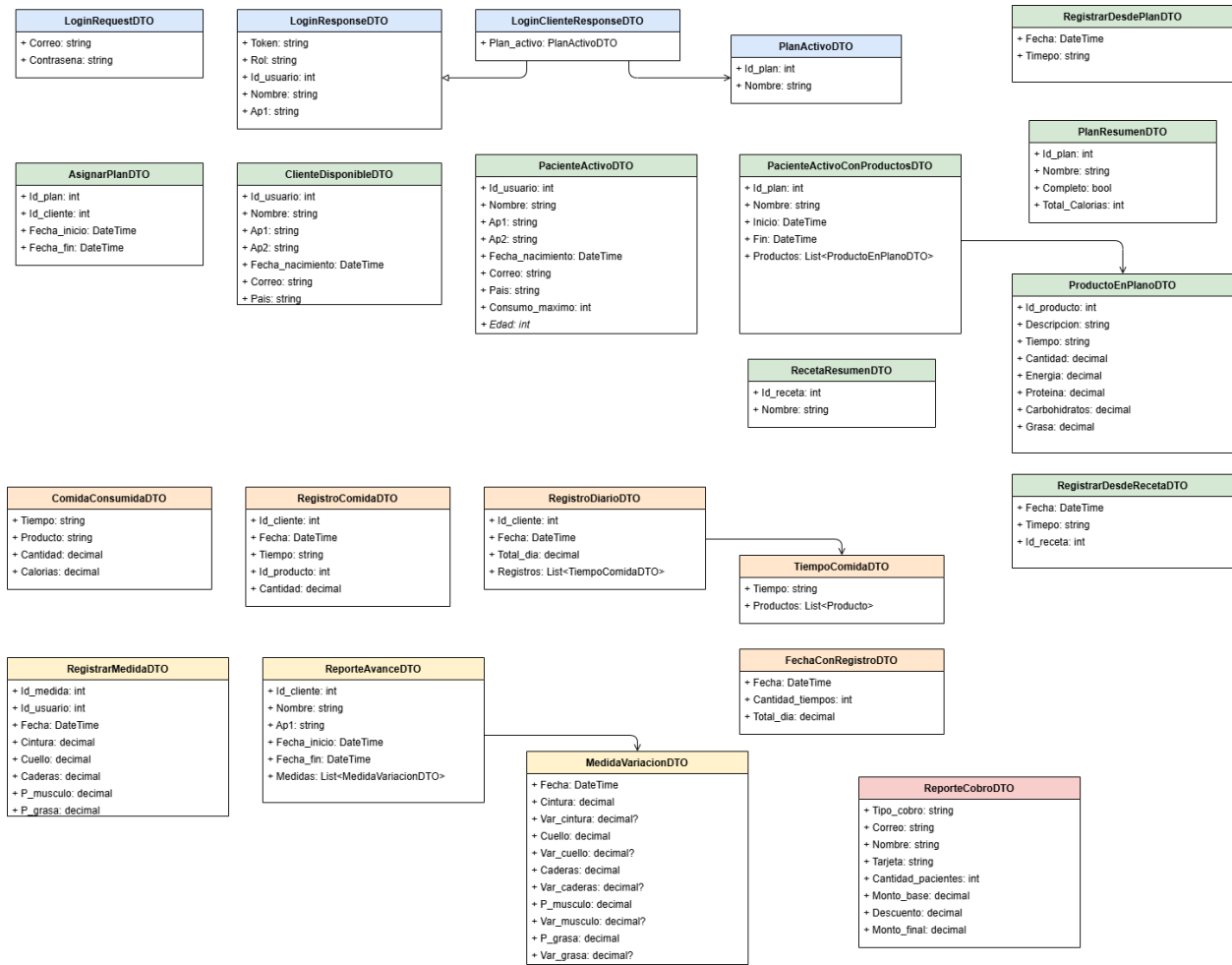


Figura 4. DTOs implementados

5. Store procedures, triggers y vistas

5.1 Store Procedures

Toda operación que involucra lógica de negocio relevante a nivel de base de datos se implementa mediante Stored Procedures en SQL Server. Ningún Stored Procedure consiste en una única sentencia SQL. En total se implementaron ocho Stored Procedures, una de ellas las cuales se detallan a continuación:

Tabla 7. Descripción de Store Procedures implementados.

Nombre	Descripción
SP_RegistrarNutricionista	Inserta en una transacción el Usuario base y los datos específicos en Nutricionista, retornando el id_usuario generado mediante un parámetro OUTPUT. Revierte ambas inserciones si alguna falla.
SP_RegistrarCliente	Inserta en una transacción el Usuario base y los datos específicos en Cliente (país, consumo máximo), retornando el id_usuario generado por OUTPUT.
sp_AprobarProducto	Valida que el producto exista y, de ser así, actualiza su Estado a 'aprobado'. Si no existe, levanta un error con RAISERROR.
sp_EliminarProducto	Elimina un producto y sus vitaminas asociadas dentro de una transacción, validando previamente su existencia.
sp_ExcesoCalorico	Calcula las calorías consumidas por un cliente en una fecha específica y las compara contra su consumo máximo, retornando un mensaje de alerta si se supera el límite.
SP — Registrar consumo desde Plan	Recibe id_cliente, fecha y tiempo de comida; busca el plan activo del cliente en PlanxCiente para esa fecha, obtiene los productos de ese tiempo de comida en ProductoxPlan, y crea automáticamente el Registro_Diario y los registros en RegistroxProducto con las cantidades del plan.

SP — Registrar consumo desde Receta	Recibe id_cliente, fecha, tiempo y id_receta; obtiene los productos de ProductoxReceta y crea el Registro_Diario junto con los registros en RegistroxProducto con las cantidades de la receta.
SP — Reporte de Cobro por Tipo de Pago	SP obligatorio según el enunciado. Cuenta los pacientes por nutricionista en ClientexNutricionista, calcula el monto base según el tipo de cobro (semanal: \$1/paciente, mensual: \$4/paciente con 5% de descuento, anual: \$52/paciente con 10% de descuento) y retorna el resultado agrupado por tipo de cobro.

5.2 Triggers

Los Triggers permiten ejecutar acciones automáticas en respuesta a eventos de modificación de datos. Se utilizan para reforzar reglas de negocio, validar restricciones adicionales y mantener la consistencia de la información sin intervención directa de la aplicación.

Tabla 8. Descripción de Triggers implementados.

Nombre	Evento / Tabla	Descripción
trg_Validar CaloriasPlan	AFTER INSERT en PlanxCiente	Al asignar un plan a un cliente, valida primero que el cliente no tenga ya un plan asignado cuyo rango de fechas se solape con el nuevo; luego calcula el total de calorías del plan (suma de Energía × Cantidad proporcional a la porción, desde ProductoxPlan y Producto) y lo compara contra el Consumo_maximo del cliente. Si el plan supera el consumo máximo permitido, o si existe traslape de fechas, revierte la transacción con RAISERROR y un mensaje descriptivo generado con FORMATMESSAGE.
trg_Revertir Producto APendiente	AFTER UPDATE en Producto	Al editar un producto, primero verifica si está siendo referenciado en ProductoxPlan, ProductoxReceta o RegistroxProducto; si está en uso, rechaza la modificación con RAISERROR y ROLLBACK. Si no está en uso y el producto estaba previamente 'aprobado' y alguno de sus valores nutricionales cambió, revierte automáticamente su Estado a 'pendiente', forzando una nueva aprobación del administrador. Incluye control de TRIGGER_NESTLEVEL() para evitar recursión cuando el propio trigger genera una actualización adicional.

6.3 Vistas (Views)

Las Vistas proporcionan consultas predefinidas que integran información de múltiples tablas en una estructura más sencilla y accesible. Facilitan la generación de reportes, la consulta de datos relevantes y la reutilización de lógica de consulta dentro del sistema.

Tabla 9. Descripción de Vistas implementadas.

Nombre	Descripción
Vista_RegistroDiario	Combina Registro_Diario, RegistroxProducto y Producto para mostrar, por cliente, fecha y tiempo de comida, cada producto consumido con su cantidad y las calorías resultantes ($\text{Energía} \times \text{Cantidad}$).
vw_ClientesSinNutricionista	Retorna los clientes que no tienen ningún nutricionista asociado en ClientexNutricionista, mediante un LEFT JOIN filtrando donde la asociación es NULL. Utilizada por el nutricionista para buscar nuevos pacientes potenciales.
vw_PacientesActivos	Lista, para cada nutricionista, sus pacientes activos junto con sus datos personales y su consumo máximo, mediante un INNER JOIN entre Usuario, Cliente y ClientexNutricionista.
vw_PlanNutricionista	Para cada plan de un nutricionista, totaliza las calorías del plan sumando $\text{Energía} \times (\text{Cantidad} / \text{Porcion})$ de cada producto asociado en ProductoxPlan, agrupando por nutricionista y plan.

La arquitectura del sistema NutriTEc se basa en un enfoque multicapa orientado a servicios, que permite separar claramente la interfaz de usuario, la lógica de negocio y el acceso a datos. El frontend está desarrollado con React y Bootstrap, la capa de servicios en C# con Entity Framework, y el despliegue se realiza en Azure.

La arquitectura está compuesta por cuatro capas principales:

Capa de Controladores: API REST desarrollada en C# con .NET, desplegada como un App Service Linux en Azure. Expone los endpoints del sistema, recibe solicitudes HTTP, aplica validaciones básicas y delega a la capa de servicios.



Capa de Servicios: concentra toda la lógica de negocio del sistema, incluyendo cálculo de totales calóricos, cobros de nutricionistas, validación de aprobaciones de productos y construcción de DTOs para el frontend.

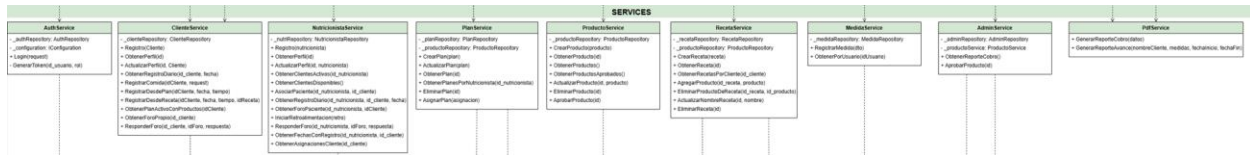


Figura 6. Capa de Servicios

Capa de Repositorios / Acceso a Datos: responsable del acceso y persistencia de datos mediante Entity Framework Core sobre Azure SQL Database en modo Serverless (región West US 3) para datos estructurados, y el driver oficial de MongoDB para .NET para la retroalimentación del foro.

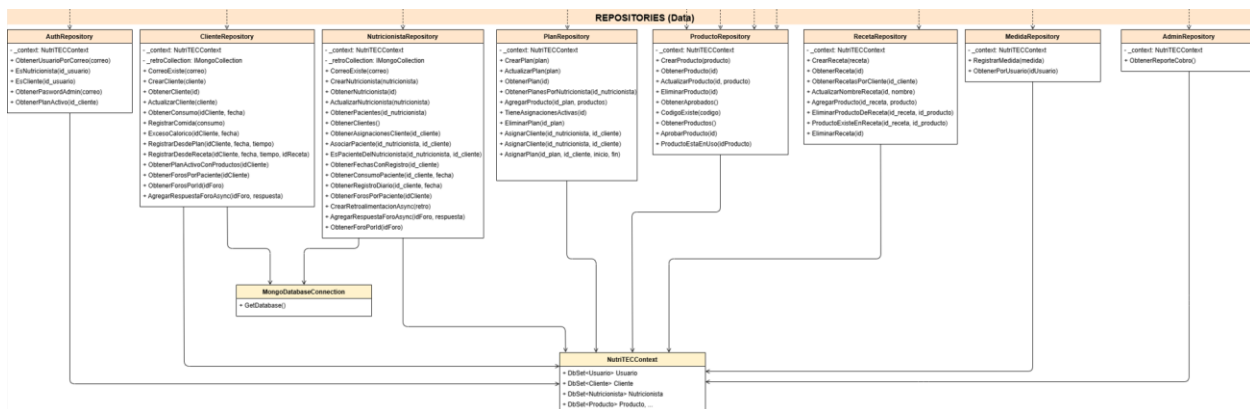


Figura 7. Capa de Repositorios

6.2 Flujo de comunicación

El flujo sigue una estructura unidireccional: la capa de presentación envía una solicitud HTTP al controlador correspondiente, quien la valida y delega a la capa de servicios. La capa de servicios aplica la lógica de negocio y consulta o actualiza datos a través de los repositorios, los cuales gestionan la comunicación con Azure SQL Server mediante Stored Procedures vía Entity Framework, o con MongoDB para el módulo de foro. La respuesta viaja de regreso en formato JSON hasta la capa de presentación.

La URL del backend no está embebida en el código fuente del frontend sino que se inyecta como variable de entorno `REACT_APP_API_URL` durante el proceso de compilación en el workflow de GitHub Actions, lo que desacopla la configuración del entorno de producción del código fuente y permite cambiar el endpoint sin modificar el repositorio.

Figura 8. Diagrama de clases

7. Problemas conocidos

Durante el desarrollo y pruebas del sistema se identificaron los siguientes problemas conocidos que no fueron resueltos dentro del alcance del proyecto:

Actualmente, el sistema presenta una limitación al registrar medidas corporales de clientes cuando el valor ingresado supera los tres dígitos. Esta situación provoca un error de desbordamiento (*overflow*) debido al tipo de dato definido en la base de datos, impidiendo almacenar correctamente valores mayores al límite establecido.

Se identificaron inconvenientes para ejecutar la aplicación desarrollada en React Native en diferentes entornos de desarrollo. Estos problemas están relacionados con configuraciones específicas del entorno, dependencias externas y archivos de compilación necesarios para la ejecución de la aplicación, lo que puede requerir ajustes manuales adicionales antes de su despliegue o prueba en nuevos dispositivos.

Los controladores de la API no implementan el atributo [Authorize], por lo que el token JWT generado en el login no es validado en los endpoints. Cualquier cliente con acceso a la URL de la API puede realizar llamadas directamente sin autenticación.

La configuración de CORS utiliza una política AllowAll que permite solicitudes desde cualquier origen. Si bien esto es funcional para el entorno de desarrollo, representa un riesgo de seguridad para un despliegue en producción.

El frontend no detecta respuestas 401 del servidor. Cuando el token JWT expira tras las 8 horas configuradas, el usuario no es redirigido al login automáticamente sino que las peticiones simplemente fallan sin retroalimentación clara.

Las contraseñas del administrador se almacenan con BCrypt pero no existe un endpoint para restablecerlas, por lo que cualquier cambio requiere intervención directa en la base de datos.

El token JWT se almacena en localStorage, lo cual es vulnerable a ataques XSS. La alternativa recomendada para producción sería el uso de cookies httpOnly.

Las fotos de perfil de los nutricionistas se almacenan como cadenas base64 directamente en la base de datos y se incluyen en cada respuesta JSON del perfil, lo que

incrementa innecesariamente el tamaño de las respuestas. Una solución más eficiente sería el uso de un servicio de almacenamiento de archivos como Azure Blob Storage.

Los endpoints que retornan listados como productos, pacientes y recetas no implementan paginación, lo que puede degradar el rendimiento conforme crezca el volumen de datos.

Cuando el trigger `trg_ValidarCaloriasPlan` rechaza una asignación por solapamiento de fechas o por exceso calórico, el frontend no maneja ese error de forma específica, mostrando únicamente un mensaje genérico al usuario.

8. Problemas encontrados

A lo largo del desarrollo del proyecto se presentaron diversos desafíos técnicos que requirieron análisis e iteración para encontrar una solución adecuada.

Uno de los problemas más recurrentes fue el manejo de contraseñas en las operaciones de actualización de perfil. Tanto para el nutricionista como para el cliente, el servicio de actualización siempre rehashaba la contraseña recibida sin verificar si había cambiado, y dado que el endpoint de obtención de perfil no retornaba la contraseña por razones de seguridad, el campo llegaba vacío al momento de guardar, corrompiendo las credenciales del usuario. La solución fue separar el cambio de contraseña en una sección independiente de la UI con validación de mínimo 8 caracteres, y en el backend verificar si el campo viene vacío para conservar el hash existente directamente desde la base de datos.

La tabla `Producto` en SQL Server tiene un trigger `AFTER UPDATE` que es incompatible con la cláusula `OUTPUT` que Entity Framework Core genera automáticamente en sus operaciones de actualización. Esto causaba un error 500 en cada intento de editar un producto. La solución fue registrar explícitamente el trigger en la configuración del `DbContext` mediante `ToTable(tb => tb.HasTrigger(...))`, lo que indica a EF que omita el uso de `OUTPUT` para esa tabla.

Durante la integración de los tres proyectos React separados en una sola aplicación unificada, se encontraron diferencias en rutas de importación, URLs de API

hardcodeadas con localhost y IDs de usuario fijos en lugar de leerlos desde localStorage. Esto requirió revisar y corregir cada página del módulo de nutricionista para adaptarla al patrón de sesión real.

Las fotos de perfil del nutricionista no se guardaban correctamente al momento del registro porque el servicio forzaba `nutricionista.Foto = null` antes de persistir, ignorando el valor enviado desde el frontend. Adicionalmente, el stored procedure `SP_RegistrarNutricionista` tenía el parámetro `@Foto` definido como `VARCHAR(500)`, insuficiente para almacenar una imagen codificada en base64, y la columna en la tabla también tenía esa misma limitación. Ambos fueron corregidos a `VARCHAR(MAX)`.

El ID del nutricionista en varias páginas del frontend se leía a nivel de módulo al momento de importar el archivo, antes de que el usuario hubiera iniciado sesión, resultando siempre en cero. La solución fue mover la lectura del localStorage al interior de cada función que realiza la petición, asegurando que el valor se lee en el momento de ejecución y no al cargar el módulo.

La integración de la capa de repositorios, que existía únicamente en una de las dos ramas del proyecto, requirió identificar y resolver dependencias cruzadas con el contexto de Entity Framework, la conexión a MongoDB Atlas y los servicios existentes antes de poder unificar ambas versiones de la API.

9. Conclusiones

El proyecto NutriTEC permitió consolidar el diseño de sistemas con bases de datos híbridas. La combinación de SQL Server para datos estructurados y MongoDB Atlas para el foro de retroalimentación demostró ser acertada, ya que la naturaleza conversacional del foro se adapta mejor a un modelo documental que a un esquema relacional rígido.

La arquitectura por capas con controladores, servicios y repositorios facilitó el mantenimiento y la depuración del sistema, permitiendo identificar y corregir errores de forma localizada sin afectar otras partes. La combinación de Entity Framework Core con stored procedures permitió aprovechar la comodidad del ORM para operaciones estándar y el control directo de SQL para lógica transaccional compleja.

Unificar los tres proyectos React en una sola aplicación con redirección por rol simplificó la experiencia del usuario y la gestión del código. Aunque la migración introdujo trabajo adicional para estandarizar el manejo de sesión y URLs, el resultado es más coherente y mantenible que tres sitios independientes.

El uso de triggers para validar restricciones de negocio como el solapamiento de planes y la reversión de estado de productos editados resultó efectivo para garantizar integridad de datos a nivel de base de datos, independientemente de la capa de aplicación.

Finalmente, la coordinación del equipo en un repositorio compartido evidenció que la planificación de ramas es tan importante como el desarrollo mismo. Los conflictos de merge entre versiones paralelas fueron el principal cuello de botella operativo del proyecto.

10. Recomendaciones

Para futuras versiones o iteraciones del sistema NutriTEc, el equipo recomienda las siguientes mejoras:

- Implementar validaciones más robustas tanto en el frontend como en la capa de servicios, especialmente para los formularios de registro de usuarios, productos y registros de consumo.
- Considerar el uso de un servicio de autenticación centralizado (como JWT con refresh tokens) para mejorar la seguridad del sistema.
- Optimizar las consultas SQL mediante índices adicionales en las columnas más frecuentemente consultadas, como fecha en RegistroDiario y id_paciente en NutricionistaPaciente.
- Explorar la posibilidad de implementar notificaciones automáticas al paciente cuando el nutricionista publica una nueva retroalimentación en el foro.
- Agregar pruebas unitarias y de integración para los Stored Procedures y la capa de servicios, reduciendo el riesgo de regresiones ante cambios futuros.
- Documentar el código fuente de manera continua durante el desarrollo, no solo al final, para garantizar coherencia entre la implementación y la documentación.

11. Bibliografía

How to Write Doc Comments for the Javadoc Tool. (2023-26-05). Recuperado de:
<http://www.oracle.com/technetwork/articles/java/index-137868.html>

C# Coding Conventions (C# Programming Guide). (2023-26-05). Recuperado de:
<https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/inside-a-program/coding-conventions>